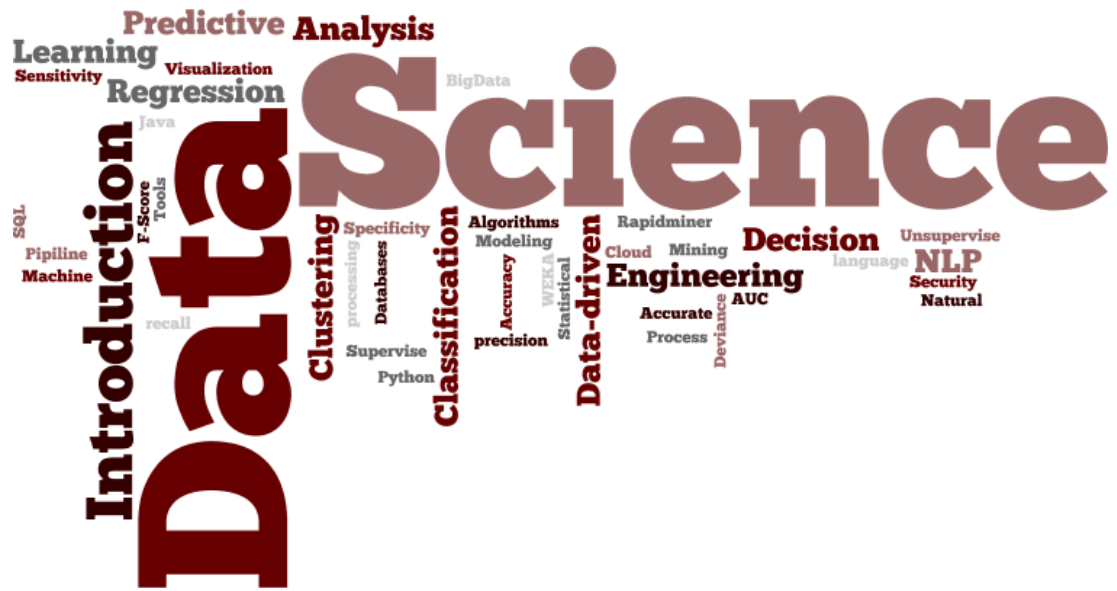




3. เครื่องมือวิทยาการข้อมูล: Python

Tools for Data Science: Python

ดำรงฤทธิ์ เศรษฐศิริโชค

3. เครื่องมือวิทยาการข้อมูล: Python

3.1 เครื่องมือสำหรับวิทยาการข้อมูล

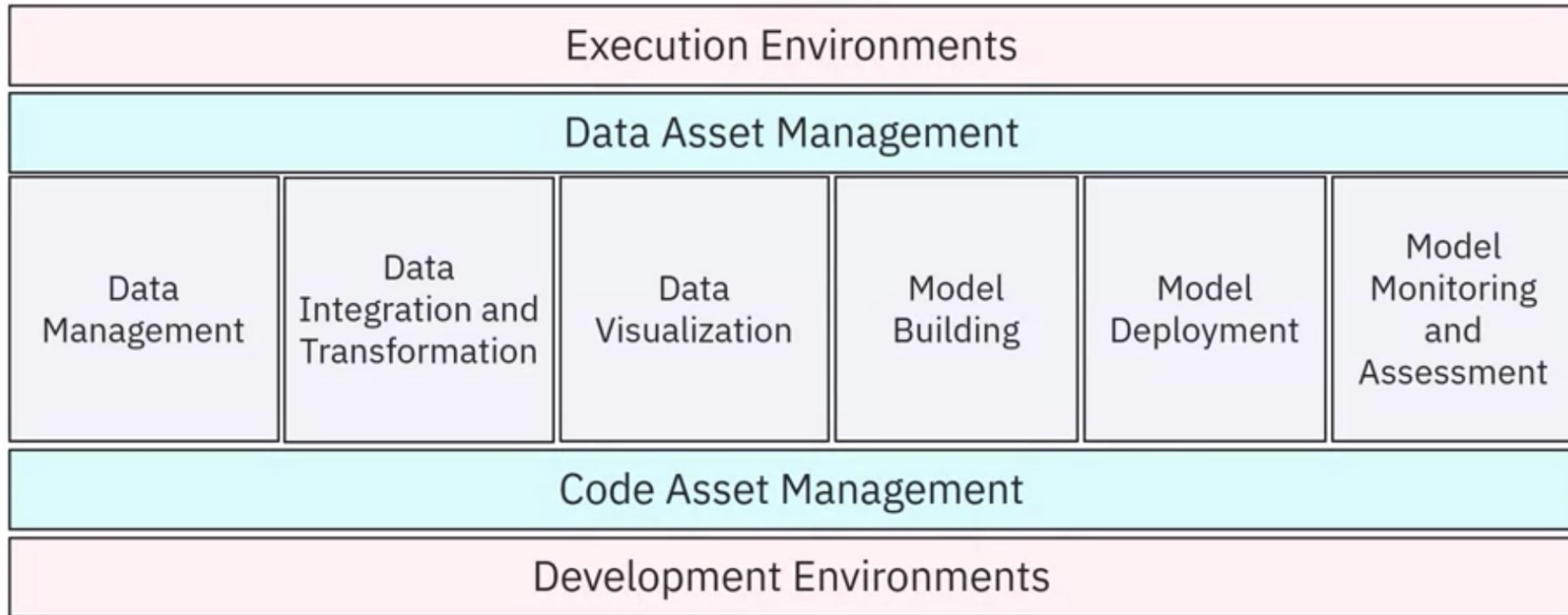
3.2 Python Packages – Numpy

3.3 Python Packages – Pandas

3.4 Python Packages – Matplotlib, Seaborn



เครื่องมือสำหรับวิทยาการข้อมูล



Open Source Tools

Commercial Tools

Data management



Open Source Tools

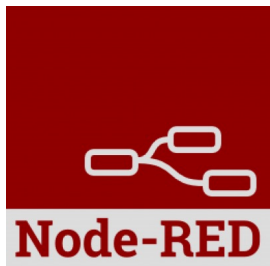


Commercial Tools

Data Integration

alteryx

talend

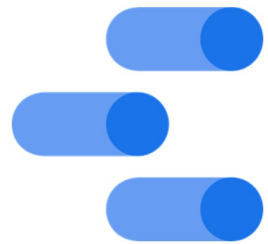


Open Source Tools

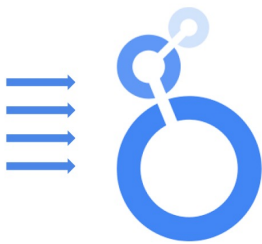


Commercial Tools

Data Visualize



Google Data Studio



Looker Studio

Open Source Tools



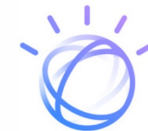
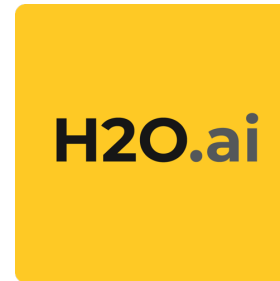
IBM Cognos Analytics

Commercial Tools

Model Building



Open Source Tools



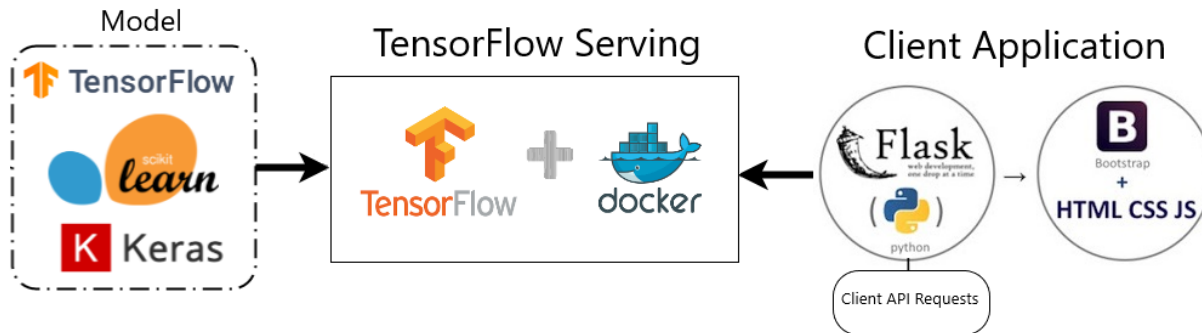
IBM **Watson**

Commercial Tools

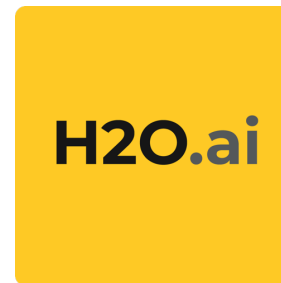
Model Deployment

 PredictionIO

 SELDON



Open Source Tools



Commercial Tools

Model monitoring and assessment



AI Explainability 360

ModelDB

A system to manage
ML models



Prometheus

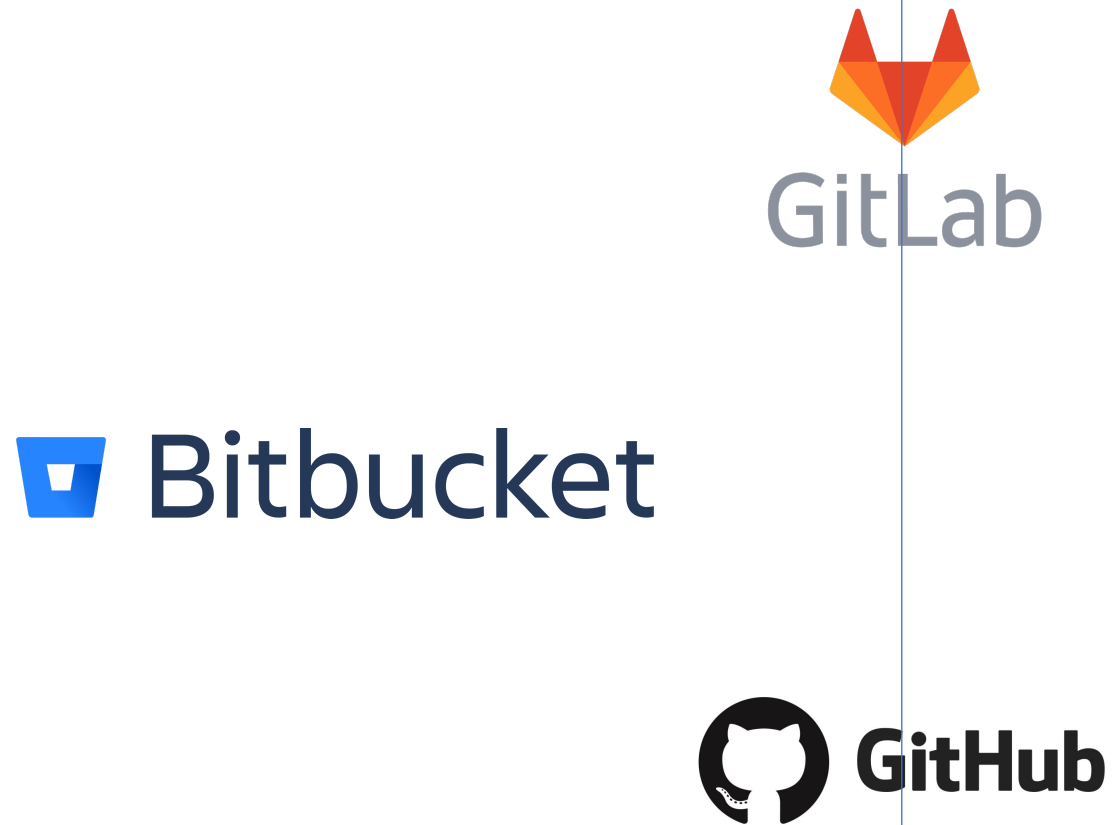
Open Source Tools



H2O.ai

Commercial Tools

Code asset management



Open Source Tools

Commercial Tools

Data asset management

- Data governance
- Data Lineage

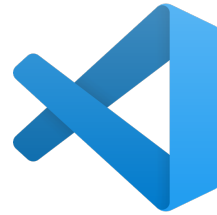


Open Source Tools



Commercial Tools

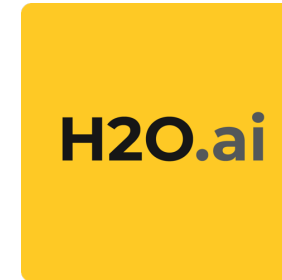
Development Environments



Cloud Based Tools for Data Science



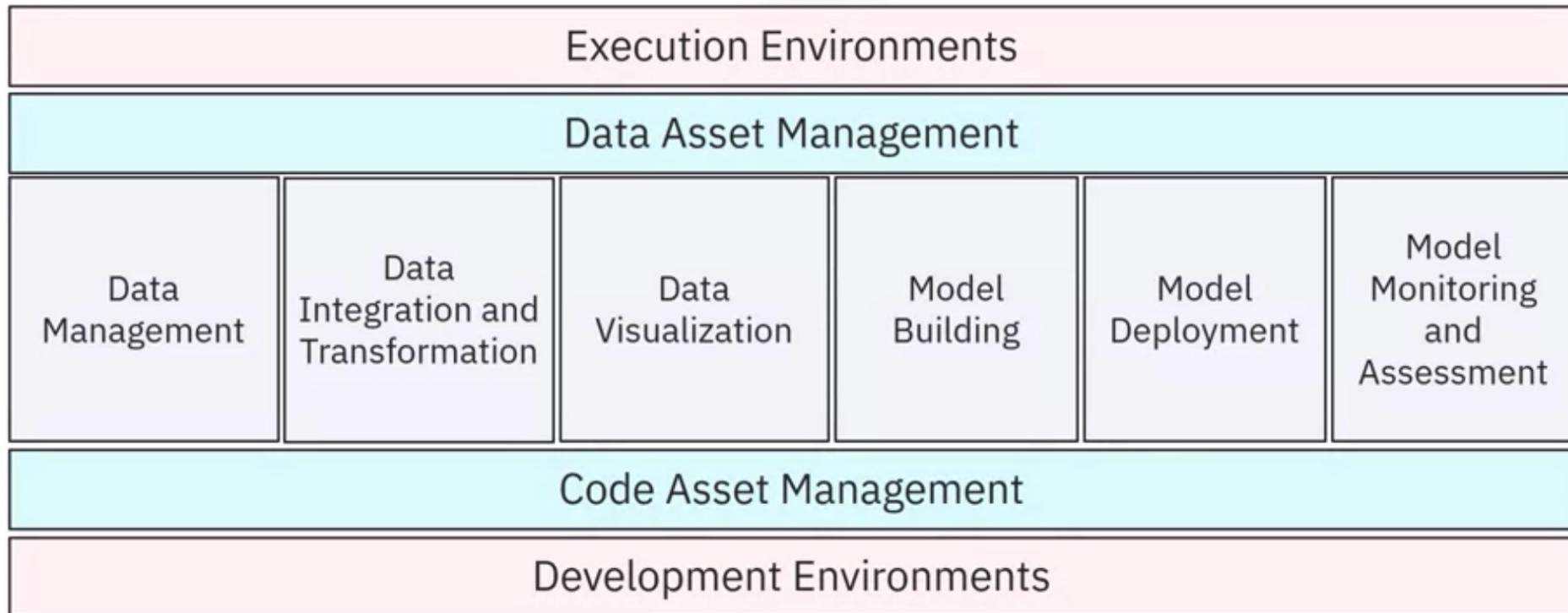
Azure Machine Learning



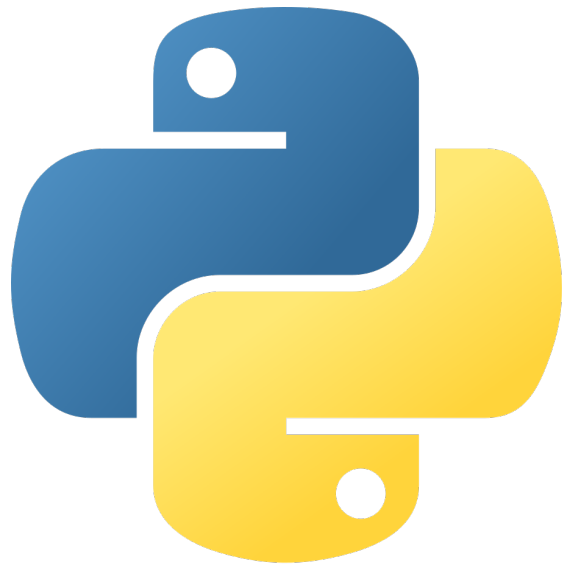
Google Cloud ML

การบ้าน

- สำรวจเครื่องมือแต่ละประเภทในงานวิทยาการข้อมูล
- ศึกษาและอธิบายคุณสมบัติของเครื่องมือประเภทละ 1 เครื่องมือ



ทบทวน Python



Data Structures and Sequences

- การวิเคราะห์ข้อมูลจำเป็นต้องใช้งานโครงสร้างข้อมูลที่เหมาะสมและมีประสิทธิภาพ
- ทบทวนโครงสร้างข้อมูล Python
 - tuple
 - list
 - dictionary
 - sequence

Tuple

- “A tuple is a fixed-length, immutable sequence of Python objects which, once assigned, cannot be changed.”
- แปลง sequence หรือ iterator เป็น tuple

```
In [2]: tup = (4, 5, 6)
```

```
In [3]: tup
```

```
Out[3]: (4, 5, 6)
```

```
In [4]: tup = 4, 5, 6
```

```
In [5]: tup
```

```
Out[5]: (4, 5, 6)
```

```
In [6]: tuple([4, 0, 2])
```

```
Out[6]: (4, 0, 2)
```

```
In [7]: tup = tuple('string')
```

```
In [8]: tup
```

```
Out[8]: ('s', 't', 'r', 'i', 'n', 'g')
```

```
In [9]: tup[0]
```

```
Out[9]: 's'
```

Tuple

- ใช้สร้างนิพจน์หรือตัวแปรที่ซับซ้อนขึ้น
- ค่าหรือ object ใดๆ ที่บรรจุภายใน tuple จะไม่สามารถแก้ไขได้ แต่สามารถเพิ่มได้

```
In [16]: tup[1].append(3)
In [17]: tup
Out[17]: ('foo', [1, 2, 3], True)
In [18]: (4, None, 'foo') + (6, 0) + ('bar',)
Out[18]: (4, None, 'foo', 6, 0, 'bar')
In [19]: ('foo', 'bar') * 4
Out[19]: ('foo', 'bar', 'foo', 'bar', 'foo', 'bar', 'foo', 'bar')
```

```
In [10]: nested_tup = (4, 5, 6), (7, 8)
In [11]: nested_tup
Out[11]: ((4, 5, 6), (7, 8))
In [12]: nested_tup[0]
Out[12]: (4, 5, 6)
In [13]: nested_tup[1]
Out[13]: (7, 8)

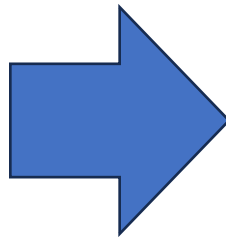
In [14]: tup = tuple(['foo', [1, 2], True])
In [15]: tup[2] = False
-----
-----

TypeError Traceback (most recent call last)
<ipython-input-15-b89d0c4ae599> in <module>
----> 1 tup[2] = False
TypeError: 'tuple' object does not support
item assignment
```

การแยก tuple

- มองภายใน tuple เป็นนิพจน์ของตัวแปร

```
tmp = a  
a = b  
b = tmp
```



```
In [20]: tup = (4, 5, 6)  
In [21]: a, b, c = tup  
In [22]: b  
Out[22]: 5
```

```
In [23]: tup = 4, 5, (6, 7)  
In [24]: a, b, (c, d) = tup  
In [25]: d  
Out[25]: 7
```

```
In [26]: a, b = 1, 2  
In [27]: a  
Out[27]: 1  
In [28]: b  
Out[28]: 2  
In [29]: b, a = a, b  
In [30]: a  
Out[30]: 2  
In [31]: b  
Out[31]: 1
```

การแยก tuple

- ใช้การวนรอบหากลำดับมีจำนวนมาก

```
In [32]: seq = [(1, 2, 3), (4, 5, 6), (7, 8, 9)]
In [33]: for a, b, c in seq:
.....: print(f'a={a}, b={b}, c={c}')
a=1, b=2, c=3
a=4, b=5, c=6
a=7, b=8, c=9
```

- ดึงค่าแถวหน้าออกมาคงเหลือส่วนอื่นไว้

```
In [34]: values = 1, 2, 3, 4, 5
In [35]: a, b, *rest = values
In [36]: a
Out[36]: 1
In [37]: b
Out[37]: 2
In [38]: rest
Out[38]: [3, 4, 5]
```

```
In [39]: a, b, *_ = values
```

underscore () for unwanted variables

Tuple methods

```
In [40]: a = (1, 2, 2, 2, 3, 4, 2)
In [41]: a.count(2)
Out[41]: 4
```

List

- เป็นตัวแปรที่ประกอบด้วยหลายค่าและสามารถเปลี่ยนแปลงได้

```
In [42]: a_list = [2, 3, 7, None]
In [43]: tup = ("foo", "bar", "baz")
In [44]: b_list = list(tup)
In [45]: b_list
Out[45]: ['foo', 'bar', 'baz']
In [46]: b_list[1] = "peekaboo"
In [47]: b_list
Out[47]: ['foo', 'peekaboo', 'baz']
```

Generator expression

```
In [48]: gen = range(10)
In [49]: gen
Out[49]: range(0, 10)
In [50]: list(gen)
Out[50]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

List: Adding and removing elements

```
In [51]: b_list.append("dwarf")
```

```
In [52]: b_list
```

```
Out[52]: ['foo', 'peekaboo', 'baz', 'dwarf']
```

```
In [53]: b_list.insert(1, "red")
```

```
In [54]: b_list
```

```
Out[54]: ['foo', 'red', 'peekaboo', 'baz', 'dwarf']
```

```
In [55]: b_list.pop(2)
```

```
Out[55]: 'peekaboo'
```

```
In [56]: b_list
```

```
Out[56]: ['foo', 'red', 'baz', 'dwarf']
```

```
In [57]: b_list.append("foo")
```

```
In [58]: b_list
```

```
Out[58]: ['foo', 'red', 'baz', 'dwarf', 'foo']
```

```
In [59]: b_list.remove("foo")
```

```
In [60]: b_list
```

```
Out[60]: ['red', 'baz', 'dwarf', 'foo']
```

List: Check if a list contains a value

```
In [61]: "dwarf" in b_list
```

```
Out[61]: True
```

```
In [62]: "dwarf" not in b_list
```

```
Out[62]: False
```

List: Concatenating and combining lists

```
In [63]: [4, None, "foo"] + [7, 8, (2, 3)]  
Out[63]: [4, None, 'foo', 7, 8, (2, 3)]
```

```
In [64]: x = [4, None, "foo"]  
In [65]: x.extend([7, 8, (2, 3)])  
In [66]: x  
Out[66]: [4, None, 'foo', 7, 8, (2, 3)]
```

extend เร็วกว่า + list

List: Sorting

```
In [67]: a = [7, 2, 5, 1, 3]
```

```
In [68]: a.sort()
```

```
In [69]: a
```

```
Out[69]: [1, 2, 3, 5, 7]
```

```
In [70]: b = ["saw", "small", "He", "foxes", "six"]
```

```
In [71]: b.sort(key=len)
```

```
In [72]: b
```

```
Out[72]: ['He', 'saw', 'six', 'small', 'foxes']
```

List: Slicing

- ตัดลำดับด้วย start:stop ภายใต้วัดดำเนินการ index

```
In [73]: seq = [7, 2, 3, 7, 5, 6, 0, 1]
In [74]: seq[1:5]
Out[74]: [2, 3, 7, 5]
```

```
In [75]: seq[3:5] = [6, 3]
In [76]: seq
Out[76]: [7, 2, 3, 6, 3, 6, 0, 1]
```

```
In [77]: seq[:5]
Out[77]: [7, 2, 3, 6, 3]
In [78]: seq[3:]
Out[78]: [6, 3, 6, 0, 1]
```

```
In [79]: seq[-4:]
Out[79]: [3, 6, 0, 1]
In [80]: seq[-6:-2]
Out[80]: [3, 6, 3, 6]
```

List: Slicing

- ใช้ step ในการดึงค่าภายใน ทุกๆ step ที่กำหนด

```
seq = [7, 2, 3, 6, 3, 6, 0, 1]
```

```
In [81]: seq[::2]
```

```
Out[81]: [7, 3, 3, 0]
```

```
In [82]: seq[::-1]
```

```
Out[82]: [1, 0, 6, 3, 6, 3, 2, 7]
```

Dictionary

- dict
- บางภาษาเรียก hash, map หรือ associative arrays
- เก็บข้อมูลคู่ key-value

```
In [83]: empty_dict = {}
In [84]: d1 = {"a": "some value", "b": [1, 2, 3, 4]}
In [85]: d1
Out[85]: {'a': 'some value', 'b': [1, 2, 3, 4]}

In [86]: d1[7] = "an integer"
In [87]: d1
Out[87]: {'a': 'some value', 'b': [1, 2, 3, 4], 7: 'an integer'}

In [88]: d1["b"]
Out[88]: [1, 2, 3, 4]

In [89]: "b" in d1
Out[89]: True
```

Dictionary

```
In [90]: d1[5] = "some value"
```

```
In [91]: d1
```

```
Out[91]:
```

```
{'a': 'some value',  
'b': [1, 2, 3, 4],  
7: 'an integer',  
5: 'some value'}
```

```
In [92]: d1["dummy"] = "another value"
```

```
In [93]: d1
```

```
Out[93]:
```

```
{'a': 'some value',  
'b': [1, 2, 3, 4],  
7: 'an integer',  
5: 'some value',  
'dummy': 'another value'}
```

```
In [94]: del d1[5]
```

```
In [95]: d1
```

```
Out[95]:
```

```
{'a': 'some value',  
'b': [1, 2, 3, 4],  
7: 'an integer',  
'dummy': 'another value'}
```

```
In [96]: ret = d1.pop("dummy")
```

```
In [97]: ret
```

```
Out[97]: 'another value'
```

```
In [98]: d1
```

```
Out[98]:
```

```
{'a': 'some value',  
'b': [1, 2, 3, 4],  
7: 'an integer'}
```

Set

- เป็นการรวมกลุ่มแบบไม่เรียงลำดับของข้อมูล unique
- ใช้กับตัวดำเนินการเกี่ยวกับ set ได้
 - Union
 - Intersection
 - Difference

```
In [124]: set([2, 2, 2, 1, 3, 3])
```

```
Out[124]: {1, 2, 3}
```

```
In [125]: {2, 2, 2, 1, 3, 3}
```

```
Out[125]: {1, 2, 3}
```

```
In [126]: a = {1, 2, 3, 4, 5}
```

```
In [127]: b = {3, 4, 5, 6, 7, 8}
```

```
In [128]: a.union(b)
```

```
Out[128]: {1, 2, 3, 4, 5, 6, 7, 8}
```

```
In [129]: a | b
```

```
Out[129]: {1, 2, 3, 4, 5, 6, 7, 8}
```

```
In [130]: a.intersection(b)
```

```
Out[130]: {3, 4, 5}
```

```
In [131]: a & b
```

```
Out[131]: {3, 4, 5}
```

Python

set operations

Function	Alternative syntax	Description
<code>a.add(x)</code>	N/A	Add element <code>x</code> to set <code>a</code>
<code>a.clear()</code>	N/A	Reset set <code>a</code> to an empty state, discarding all of its elements
<code>a.remove(x)</code>	N/A	Remove element <code>x</code> from set <code>a</code>
<code>a.pop()</code>	N/A	Remove an arbitrary element from set <code>a</code> , raising <code>KeyError</code> if the set is empty
<code>a.union(b)</code>	<code>a b</code>	All of the unique elements in <code>a</code> and <code>b</code>
<code>a.update(b)</code>	<code>a = b</code>	Set the contents of <code>a</code> to be the union of the elements in <code>a</code> and <code>b</code>
<code>a.intersection(b)</code>	<code>a & b</code>	All of the elements in <i>both</i> <code>a</code> and <code>b</code>
<code>a.intersection_update(b)</code>	<code>a &= b</code>	Set the contents of <code>a</code> to be the intersection of the elements in <code>a</code> and <code>b</code>
<code>a.difference(b)</code>	<code>a - b</code>	The elements in <code>a</code> that are not in <code>b</code>
<code>a.difference_update(b)</code>	<code>a -= b</code>	Set <code>a</code> to the elements in <code>a</code> that are not in <code>b</code>
<code>a.symmetric_difference(b)</code>	<code>a ^ b</code>	All of the elements in either <code>a</code> or <code>b</code> but <i>not both</i>
<code>a.symmetric_difference_update(b)</code>	<code>a ^= b</code>	Set <code>a</code> to contain the elements in either <code>a</code> or <code>b</code> but <i>not both</i>
<code>a.issubset(b)</code>	<code><=</code>	True if the elements of <code>a</code> are all contained in <code>b</code>
<code>a.issuperset(b)</code>	<code>>=</code>	True if the elements of <code>b</code> are all contained in <code>a</code>
<code>a.isdisjoint(b)</code>	N/A	True if <code>a</code> and <code>b</code> have no elements in common

Set

```
In [132]: c = a.copy()
In [133]: c |= b
In [134]: c
Out[134]: {1, 2, 3, 4, 5, 6, 7, 8}
In [135]: d = a.copy()
In [136]: d &= b
In [137]: d
Out[137]: {3, 4, 5}
```

```
In [138]: my_data = [1, 2, 3, 4]
In [139]: my_set = {tuple(my_data)}
In [140]: my_set
Out[140]: {(1, 2, 3, 4)}

In [141]: a_set = {1, 2, 3, 4, 5}
In [142]: {1, 2, 3}.issubset(a_set)
Out[142]: True
In [143]: a_set.issuperset({1, 2, 3})
Out[143]: True

In [144]: {1, 2, 3} == {3, 2, 1}
Out[144]: True
```

Function

```
def function_without_return(x):  
    print(x)
```

```
def function_with_return(x):  
    return(x-1)
```

- Function สามารถ return มากกว่า 1 ค่า

```
def f():  
    a = 5  
    b = 6  
    c = 7  
    return a, b, c  
a, b, c = f()
```

```
def f():  
    a = 5  
    b = 6  
    c = 7  
    return {"a" : a, "b" : b, "c" : c}
```

ตัวอย่างการออกแบบ function ทำความสะอาดข้อมูล

```
In [193]: states = [" Alabama ", "Georgia!", "Georgia", "georgia", "FlOrIda",  
.....: "south carolina##", "West virginia?"]
```

```
import re  
def clean_strings(strings):  
    result = []  
    for value in strings:  
        value = value.strip()  
        value = re.sub("[!#?]", "", value)  
        value = value.title()  
        result.append(value)  
    return result
```

```
In [195]: clean_strings(states)  
Out[195]:  
['Alabama',  
'Georgia',  
'Georgia',  
'Georgia',  
'Florida',  
'South Carolina',  
'West Virginia']
```

Anonymous (Lambda) Functions

- ฟังก์ชัน anonymous หรือเรียกว่า lambda
- ฟังก์ชันที่สามารถรับ argument เป็นฟังก์ชันได้

```
def short_function(x):  
    return x * 2
```

```
equiv_anon = lambda x: x * 2
```

```
def apply_to_list(some_list, f):  
    return [f(x) for x in some_list]
```

```
In [202]: ints = [4, 0, 1, 5, 6]
```

```
In [203]: apply_to_list(ints, lambda x: x * 2)
```

```
Out[203]: [8, 0, 2, 10, 12]
```

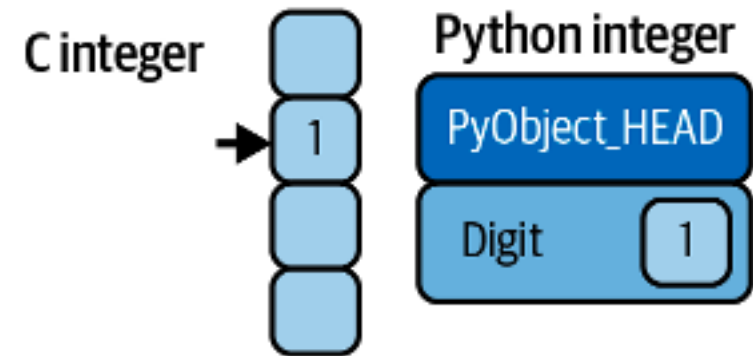
Numpy



Python Integer เป็นมากกว่า Integer

- ob_refcnt – reference count ช่วยในเรื่องจัดการหน่วยความจำ
 - ob_type เข้ารหัสชนิดตัวแปร
 - ob_size ขนาดของสมาชิกของข้อมูล
 - ob_digit เนื้อข้อมูลตัวเลข
-
- ทำให้ Python สามารถจัดการตัวแปรและบริหารหน่วยความจำแบบ dynamic ได้

```
struct _longobject {  
    long ob_refcnt;  
    PyObject *ob_type;  
    size_t ob_size;  
    long ob_digit[1];  
};
```



The difference between C and Python integers

Python List เป็นมากกว่า List

```
In [1]: L = list(range(10))
```

```
L
```

```
Out[1]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
In [2]: type(L[0])
```

```
Out[2]: int
```

```
In [3]: L2 = [str(c) for c in L]
```

```
L2
```

```
Out[3]: ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
```

```
In [4]: type(L2[0])
```

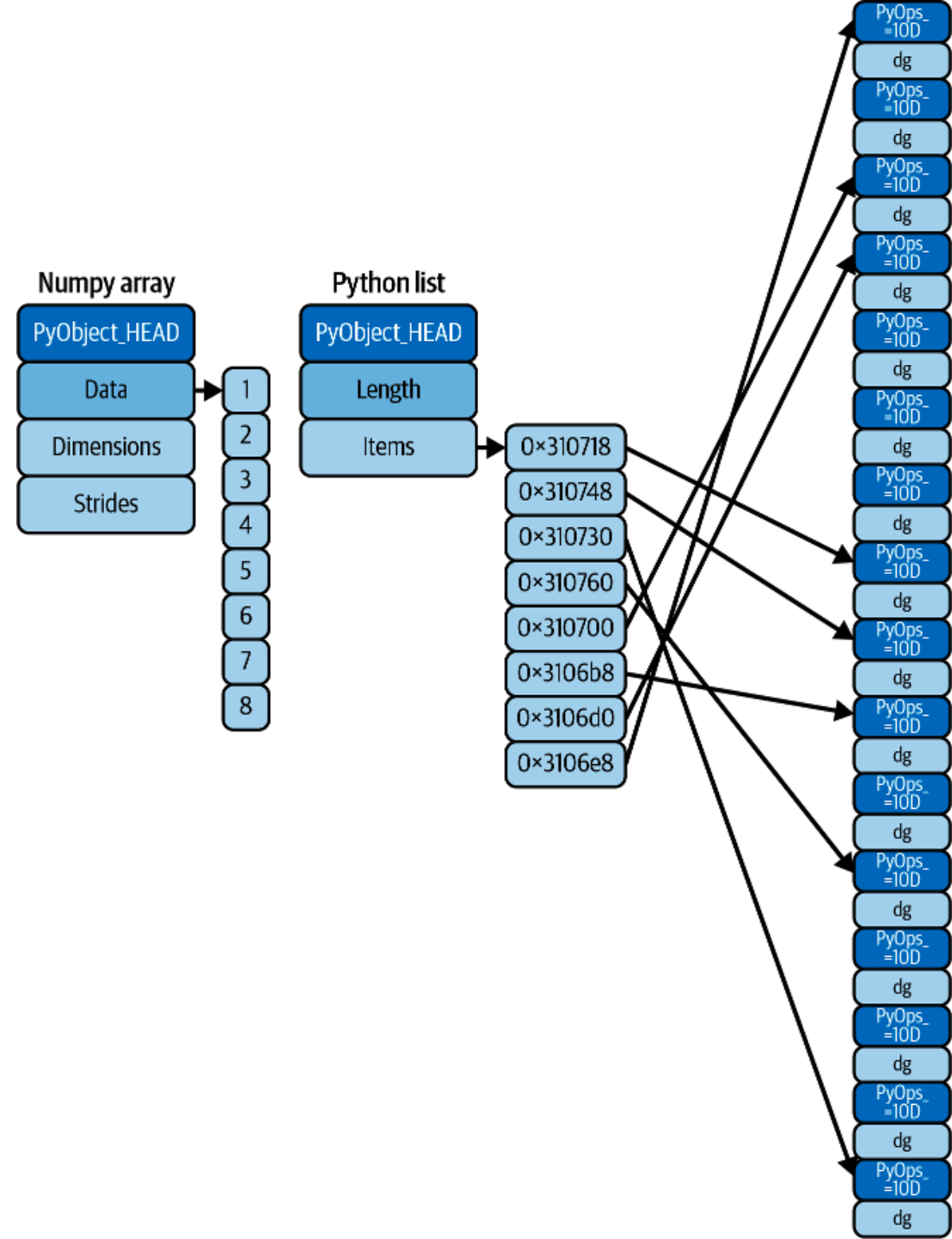
```
Out[4]: str
```

```
In [5]: L3 = [True, "2", 3.0, 4]  
         [type(item) for item in L3]
```

```
Out[5]: [bool, str, float, int]
```

Python List เป็นมากกว่า List

- ประโยชน์ของ Python list ตัว object สามารถเป็นข้อมูลชนิดใดก็ได้



Fixed-Type Arrays in Python

- การสร้าง array แบบระบุชนิด
- 'i' แทน integer

```
In [6]: import array
        L = list(range(10))
        A = array.array('i', L)
        A
Out[6]: array('i', [0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

- นำไปสู่การใช้ Numpy array เพื่อเพิ่มประสิทธิภาพด้านการดำเนินการเกี่ยวกับข้อมูล
- Numpy array จะปรับระดับชนิดข้อมูลให้เหมือนกันอัตโนมัติ

```
In [7]: import numpy as np
In [8]: # Integer array
        np.array([1, 4, 2, 5, 3])
Out[8]: array([1, 4, 2, 5, 3])

In [9]: np.array([3.14, 4, 2, 3])
Out[9]: array([3.14, 4. , 2. , 3. ])
```

Numpy array

```
In [10]: np.array([1, 2, 3, 4], dtype=np.float32)
```

```
Out[10]: array([1., 2., 3., 4.], dtype=float32)
```

```
In [11]: # Nested lists result in multidimensional arrays  
np.array([range(i, i + 3) for i in [2, 4, 6]])
```

```
Out[11]: array([[2, 3, 4],  
               [4, 5, 6],  
               [6, 7, 8]])
```

การสร้าง array จากจุดเริ่มต้น

```
In [12]: # Create a length-10 integer array filled with 0s  
         np.zeros(10, dtype=int)
```

```
Out[12]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

```
In [13]: # Create a 3x5 floating-point array filled with 1s  
         np.ones((3, 5), dtype=float)
```

```
Out[13]: array([[1., 1., 1., 1., 1.],  
                [1., 1., 1., 1., 1.],  
                [1., 1., 1., 1., 1.]])
```

```
In [14]: # Create a 3x5 array filled with 3.14  
         np.full((3, 5), 3.14)
```

```
Out[14]: array([[3.14, 3.14, 3.14, 3.14, 3.14],  
                [3.14, 3.14, 3.14, 3.14, 3.14],  
                [3.14, 3.14, 3.14, 3.14, 3.14]])
```

```
In [15]: # Create an array filled with a linear sequence
# starting at 0, ending at 20, stepping by 2
# (this is similar to the built-in range function)
np.arange(0, 20, 2)
```

```
Out[15]: array([ 0, 2, 4, 6, 8, 10, 12, 14, 16, 18])
```

```
In [16]: # Create an array of five values evenly spaced between 0 and 1
np.linspace(0, 1, 5)
```

```
Out[16]: array([0. , 0.25, 0.5 , 0.75, 1. ])
```

```
In [17]: # Create a 3x3 array of uniformly distributed
# pseudorandom values between 0 and 1
np.random.random((3, 3))
```

```
Out[17]: array([[0.09610171, 0.88193001, 0.70548015],
                [0.35885395, 0.91670468, 0.8721031 ],
                [0.73237865, 0.09708562, 0.52506779]])
```

```
In [18]: # Create a 3x3 array of normally distributed pseudorandom
# values with mean 0 and standard deviation 1
```

```
np.random.normal(0, 1, (3, 3))
```

```
Out[18]: array([[ -0.46652655, -0.59158776, -1.05392451],
               [-1.72634268,  0.03194069, -0.51048869],
               [ 1.41240208,  1.77734462, -0.43820037]])
```

```
In [19]: # Create a 3x3 array of pseudorandom integers in the interval [0, 10)
np.random.randint(0, 10, (3, 3))
```

```
Out[19]: array([[4, 3, 8],
               [6, 5, 0],
               [1, 1, 4]])
```

```
In [20]: # Create a 3x3 identity matrix
np.eye(3)
```

```
Out[20]: array([[1., 0., 0.],
               [0., 1., 0.],
               [0., 0., 1.]])
```

```
In [21]: # Create an uninitialized array of three integers; the values will be
# whatever happens to already exist at that memory location
np.empty(3)
```

```
Out[21]: array([1., 1., 1.])
```

Table 4-1. Standard NumPy data types

Data type	Description
bool_	Boolean (True or False) stored as a byte
int_	Default integer type (same as C long; normally either int64 or int32)
intc	Identical to C int (normally int32 or int64)
intp	Integer used for indexing (same as C ssize_t; normally either int32 or int64)
int8	Byte (−128 to 127)
int16	Integer (−32768 to 32767)
int32	Integer (−2147483648 to 2147483647)
int64	Integer (−9223372036854775808 to 9223372036854775807)
uint8	Unsigned integer (0 to 255)
uint16	Unsigned integer (0 to 65535)
uint32	Unsigned integer (0 to 4294967295)
uint64	Unsigned integer (0 to 18446744073709551615)
float_	Shorthand for float64
float16	Half-precision float: sign bit, 5 bits exponent, 10 bits mantissa
float32	Single-precision float: sign bit, 8 bits exponent, 23 bits mantissa
float64	Double-precision float: sign bit, 11 bits exponent, 52 bits mantissa
complex_	Shorthand for complex128
complex64	Complex number, represented by two 32-bit floats
complex128	Complex number, represented by two 64-bit floats

```
np.zeros(10, dtype='int16')
```

Or

```
np.zeros(10, dtype=np.int16)
```

2D Array Slicing

```
In [22]: x
Out[22]: array([[12, 1, 3, 7],
               [ 4, 0, 2, 3],
               [ 0, 0, 6, 9]])
```

```
In [23]: x[:2, :3] # first two rows & three columns
Out[23]: array([[12, 1, 3],
               [ 4, 0, 2]])
```

```
In [24]: x[:3, ::2] # three rows, every second column
Out[24]: array([[12, 3],
               [ 4, 2],
               [ 0, 6]])
```

```
In [25]: x[::-1, ::-1] # all rows & columns, reversed
Out[25]: array([[ 9, 6, 0, 0],
               [ 3, 2, 0, 4],
               [ 7, 3, 1, 12]])
```

Subarray = View

- เมื่อเราตัดส่วน Numpy array จะได้รับ view
- เมื่อเปลี่ยนค่าใน view จะทำให้ต้นฉบับเปลี่ยนด้วย
- หากต้องการทำสำเนา (copy) ต้องใช้ “np.copy()”

```
x_sub_copy = x[:,2, :2].copy()
print(x_sub_copy)
```

```
[[99 1]
 [ 4 0]]
```

```
In [29]: print(x)
Out[29]: [[12 1 3 7]
 [ 4 0 2 3]
 [ 0 0 6 9]]
```

```
In [30]: x_sub = x[:,2, :2]
print(x_sub)
Out[30]: [[12 1]
 [ 4 0]]
```

```
In [31]: x_sub[0, 0] = 99
print(x_sub)
Out[31]: [[99 1]
 [ 4 0]]
```

```
In [32]: print(x)
Out[32]: [[99 1 3 7]
 [ 4 0 2 3]
 [ 0 0 6 9]]
```

Reshaping of arrays

```
In [36]: grid = np.arange(1, 10).reshape(3, 3)
          print(grid)
```

```
Out[36]: [[1 2 3]
          [4 5 6]
          [7 8 9]]
```

```
In [37]: x = np.array([1, 2, 3])
          x.reshape((1, 3)) # row vector via reshape
```

```
Out[37]: array([[1, 2, 3]])
```

```
In [38]: x.reshape((3, 1)) # column vector via reshape
```

```
Out[38]: array([[1],
                [2],
                [3]])
```

```
In [39]: x[np.newaxis, :] # row vector via newaxis
```

```
Out[39]: array([[1, 2, 3]])
```

```
In [40]: x[:, np.newaxis] # column vector via newaxis
```

```
Out[40]: array([[1],
                [2],
                [3]])
```

Concatenation of Arrays

```
In [41]: x = np.array([1, 2, 3])  
         y = np.array([3, 2, 1])  
         np.concatenate([x, y])
```

```
Out[41]: array([1, 2, 3, 3, 2, 1])
```

```
In [43]: grid = np.array([[1, 2, 3],  
                          [4, 5, 6]])
```

```
In [44]: # concatenate along the first axis  
         np.concatenate([grid, grid])
```

```
Out[44]: array([[1, 2, 3],  
                [4, 5, 6],  
                [1, 2, 3],  
                [4, 5, 6]])
```

```
In [45]: # concatenate along the second axis (zero-indexed)  
         np.concatenate([grid, grid], axis=1)
```

```
Out[45]: array([[1, 2, 3, 1, 2, 3],  
                [4, 5, 6, 4, 5, 6]])
```

Concatenation of Arrays

- array มีขนาดไม่เท่ากัน
- `vstack()` เชื่อมกันแนวตั้ง
- `hstack()` เชื่อมกันแนวนอน

```
In [46]: # vertically stack the arrays  
         np.vstack([x, grid])
```

```
Out[46]: array([[1, 2, 3],  
               [1, 2, 3],  
               [4, 5, 6]])
```

```
In [47]: # horizontally stack the arrays  
         y = np.array([[99],  
                       [99]])  
         np.hstack([grid, y])
```

```
Out[47]: array([[ 1,  2,  3, 99],  
               [ 4,  5,  6, 99]])
```

Splitting of Arrays

```
In [48]: x = [1, 2, 3, 99, 99, 3, 2, 1]
         x1, x2, x3 = np.split(x, [3, 5])
         print(x1, x2, x3)
```

```
Out[48]: [1 2 3] [99 99] [3 2 1]
```

```
In [49]: grid = np.arange(16).reshape((4, 4))
         grid
```

```
Out[49]: array([[ 0,  1,  2,  3],
                [ 4,  5,  6,  7],
                [ 8,  9, 10, 11],
                [12, 13, 14, 15]])
```

Splitting of Arrays

```
In [50]: upper, lower = np.vsplit(grid, [2])  
         print(upper)  
         print(lower)
```

```
Out[50]: [[0 1 2 3]  
          [4 5 6 7]]  
         [[ 8 9 10 11]  
          [12 13 14 15]]
```

```
In [51]: left, right = np.hsplit(grid, [2])  
         print(left)  
         print(right)
```

```
Out[51]: [[ 0 1]  
          [ 4 5]  
          [ 8 9]  
          [12 13]]  
         [[ 2 3]  
          [ 6 7]  
          [10 11]  
          [14 15]]
```

การคำนวณ NumPy Array



การคำนวณ NumPy Array

- การประมวลผลหรือคำนวณด้วยโปรแกรมวนรอบจะทำให้การทำงานช้า
- จากตัวอย่าง หากข้อมูลมีขนาดใหญ่จะใช้เวลาหลายวินาที
- Numpy ใช้การประมวลผลแบบ vector ได้ ทำให้คำนวณได้เร็วขึ้น

```
In [1]: import numpy as np
rng = np.random.default_rng(seed=1701)
def compute_reciprocals(values):
    output = np.empty(len(values))
    for i in range(len(values)):
        output[i] = 1.0 / values[i]
    return output
```

```
values = rng.integers(1, 10, size=5)
compute_reciprocals(values)
Out[1]: array([0.11111111, 0.25 , 1. ,
               0.33333333, 0.125 ])
```

```
In [2]: big_array = rng.integers(1, 100, size=1000000)
%timeit compute_reciprocals(big_array)
Out[2]: 2.61 s ± 192 ms per loop
(mean ± std. dev. of 7 runs, 1 loop each)
```

Universal Functions: Ufuncs

- Vectorized operations ของ Numpy
พัฒนาบน ufuncs
- ดำเนินการได้ทั้ง 1 input และ 2 input

```
In [8]: x = [0 1 2 3]
        print("-x = ", -x)
        print("x ** 2 = ", x ** 2)
        print("x % 2 = ", x % 2)

Out[8]: -x = [ 0 -1 -2 -3]
        x ** 2 = [0 1 4 9]
        x % 2 = [0 1 0 1]
```

```
In [9]: -(0.5*x + 1) ** 2
Out[9]: array([-1. , -2.25, -4. , -6.25])
```

```
In [10]: np.add(x, 2)
Out[10]: array([2, 3, 4, 5])
```

Table 6-1. Arithmetic operators implemented in NumPy

Operator	Equivalent ufunc	Description
+	np.add	Addition (e.g., $1 + 1 = 2$)
-	np.subtract	Subtraction (e.g., $3 - 2 = 1$)
-	np.negative	Unary negation (e.g., -2)
*	np.multiply	Multiplication (e.g., $2 * 3 = 6$)
/	np.divide	Division (e.g., $3 / 2 = 1.5$)
//	np.floor_divide	Floor division (e.g., $3 // 2 = 1$)
**	np.power	Exponentiation (e.g., $2 ** 3 = 8$)
%	np.mod	Modulus/remainder (e.g., $9 \% 4 = 1$)

การกำหนดข้อมูลออก Ufuncs

```
In [24]: x = np.arange(5)
         y = np.empty(5)
         np.multiply(x, 10, out=y)
         print(y)
```

```
Out[24]: [ 0., 10., 20., 30., 40.]
```

```
In [25]: y = np.zeros(10)
         np.power(2, x, out=y[::2])    **Slicing
         print(y)
```

```
Out[25]: [ 1., 0., 2., 0., 4., 0., 8., 0., 16., 0.]
```

การหาผลรวม Numpy array

- เปรียบเทียบการประมวลผลหาผลรวมกับ function sum

```
In [4]: big_array = rng.random(1000000)
        %timeit sum(big_array)
        %timeit np.sum(big_array)
Out[4]: 89.9 ms ± 233 µs per loop
        (mean ± std. dev. of 7 runs, 10 loops each)
        521 µs ± 8.37 µs per loop
        (mean ± std. dev. of 7 runs, 1000 loops each)
```

ค่าสูงสุด – ต่ำสุด Numpy array

```
rng = np.random.default_rng() #Random Number Generator
```

```
In [9]: M = rng.integers(0, 10, (3, 4)) # 0-10 3rows, 4 columns  
        print(M)
```

```
Out[9]: [[0 3 1 2]  
         [1 9 7 0]  
         [4 8 3 7]]
```

```
In [11]: M.min(axis=0)
```

```
Out[11]: array([0, 3, 1, 0])
```

```
In [12]: M.max(axis=1)
```

```
Out[12]: array([3, 9, 8])
```

Table 7-1. Aggregation functions available in NumPy

Function name	NaN-safe version	Description
<code>np.sum</code>	<code>np.nansum</code>	Compute sum of elements
<code>np.prod</code>	<code>np.nanprod</code>	Compute product of elements
<code>np.mean</code>	<code>np.nanmean</code>	Compute mean of elements
<code>np.std</code>	<code>np.nanstd</code>	Compute standard deviation
<code>np.var</code>	<code>np.nanvar</code>	Compute variance
<code>np.min</code>	<code>np.nanmin</code>	Find minimum value
<code>np.max</code>	<code>np.nanmax</code>	Find maximum value
<code>np.argmin</code>	<code>np.nanargmin</code>	Find index of minimum value
<code>np.argmax</code>	<code>np.nanargmax</code>	Find index of maximum value
<code>np.median</code>	<code>np.nanmedian</code>	Compute median of elements
<code>np.percentile</code>	<code>np.nanpercentile</code>	Compute rank-based statistics of elements
<code>np.any</code>	N/A	Evaluate whether any elements are true
<code>np.all</code>	N/A	Evaluate whether all elements are true

Example: What Is the Average Height of US Presidents?

Numpy Arrays: Broadcasting

```
In [6]: a = np.arange(3)
        b = np.arange(3)[: , np.newaxis]
        print(a)
        print(b)

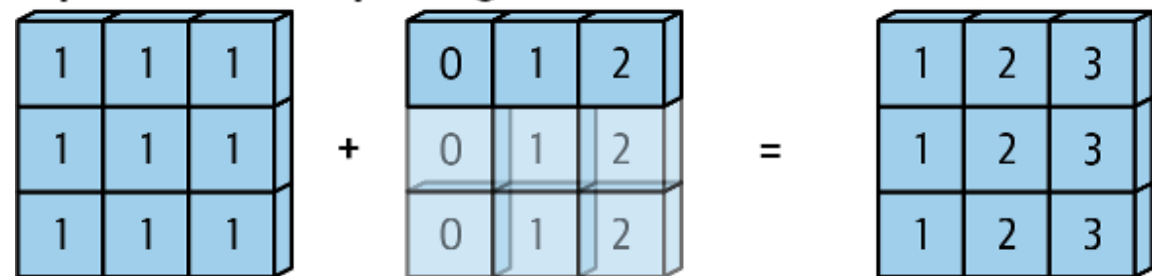
Out[6]: [0 1 2]
        [[0]
         [1]
         [2]]

In [7]: a + b
Out[7]: array([[0, 1, 2],
               [1, 2, 3],
               [2, 3, 4]])
```

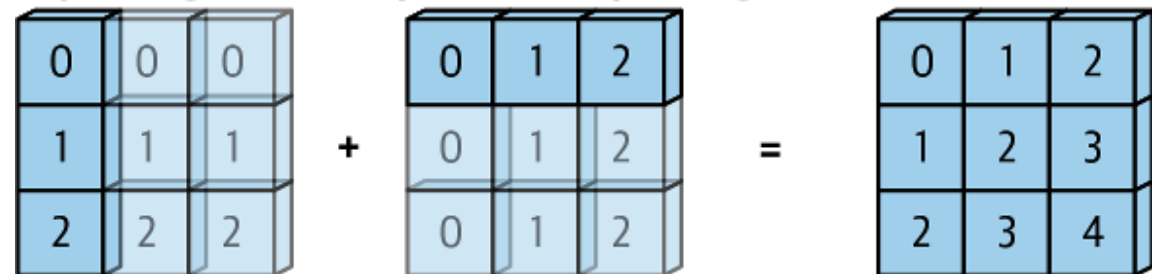
`np.arange(3)+5`



`np.ones((3,3))+np.arange(3)`



`np.arange(3).reshape((3,1))+np.arange(3)`



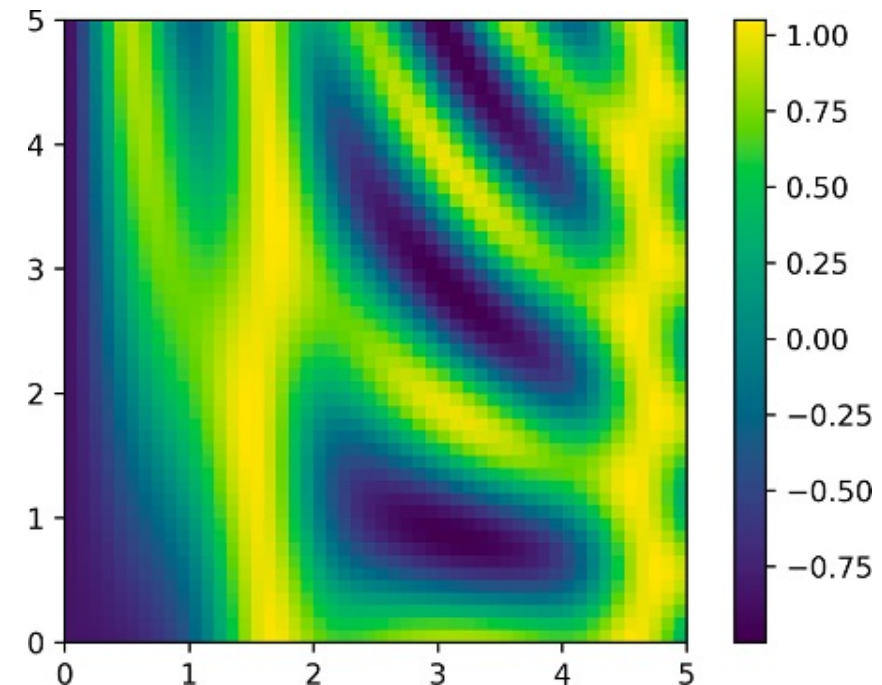
ทดลอง 2-D plot

- $z = f(x, y)$

```
In [21]: # x and y have 50 steps from 0 to 5
x = np.linspace(0, 5, 50)
y = np.linspace(0, 5, 50)[:, np.newaxis]
z = np.sin(x) ** 10 + np.cos(10 + y * x) * np.cos(x)

In [22]: %matplotlib inline
import matplotlib.pyplot as plt

In [23]: plt.imshow(z, origin='lower', extent=[0, 5, 0, 5])
plt.colorbar();
```



ใช้ Index ช่วยงาน

- การระบุตำแหน่งช่วยให้เข้าถึงได้เร็วขึ้น
- เรียกว่า *fancy* หรือ *vectorized indexing*

```
In [2]: [x[3], x[7], x[2]]  
Out[2]: [30, 15, 9]
```



```
In [1]: import numpy as np  
rng = np.random.default_rng(seed=1701)  
x = rng.integers(100, size=10)  
print(x)  
Out[1]: [90 40 9 30 80 67 39 15 33 79]
```

```
In [3]: ind = [3, 7, 4]  
x[ind]  
Out[3]: array([30, 15, 80])
```

```
In [4]: ind = np.array([[3, 7],  
                        [4, 5]])  
x[ind]  
Out[4]: array([[30, 15],  
               [80, 67]])
```

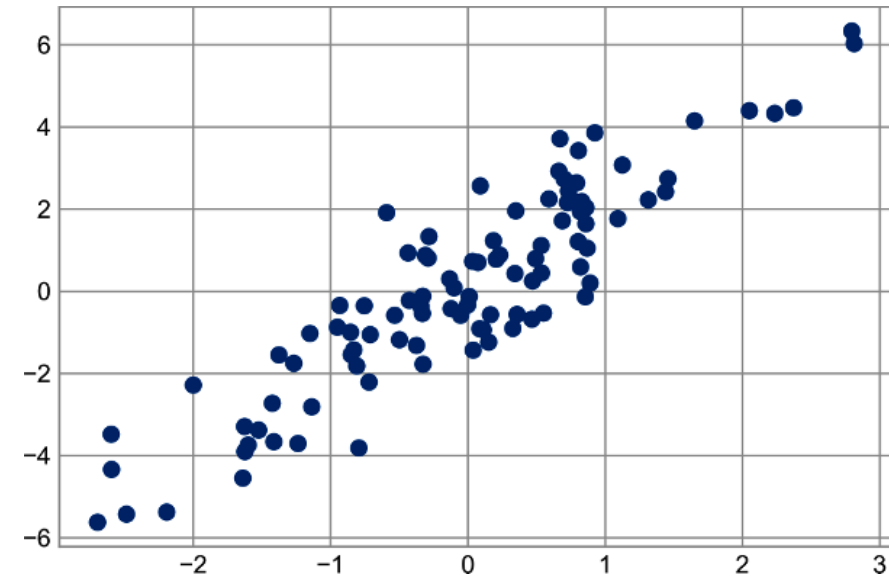
Example: Selecting Random Points (Scatter plot)

```
In [13]: mean = [0, 0]
         cov = [[1, 2],
               [2, 5]]

         X = rng.multivariate_normal(mean, cov, 100)
         X.shape

Out[13]: (100, 2)
```

```
In [14]: %matplotlib inline
         import matplotlib.pyplot as plt
         plt.style.use('seaborn-whitegrid')
         plt.scatter(X[:, 0], X[:, 1]);
```



Example: Selecting Random Points (Scatter plot)

- Random selection among points

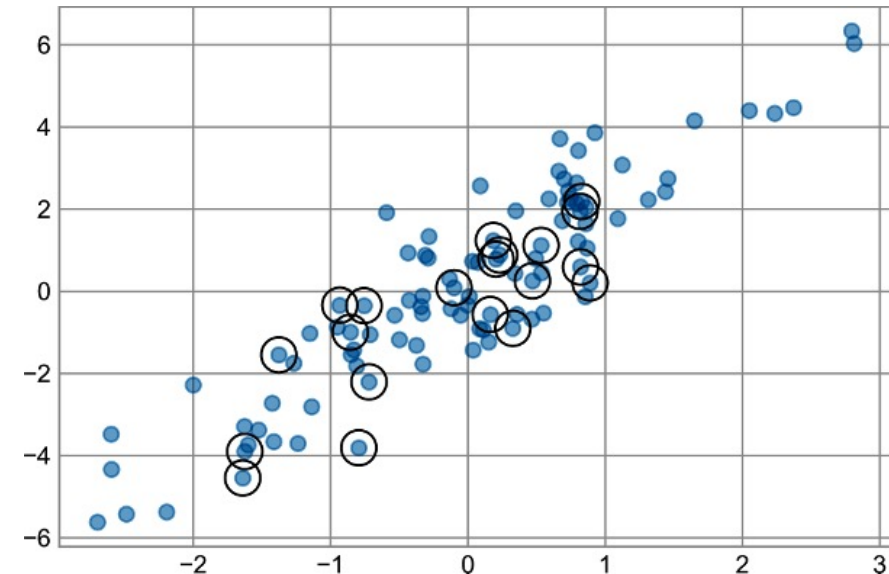
```
In [15]: indices = np.random.choice(X.shape[0], 20, replace=False)
         indices
```

```
Out[15]: array([82, 84, 10, 55, 14, 33, 4, 16, 34, 92,
                99, 64, 8, 76, 68, 18, 59, 80, 87, 90])
```

```
In [16]: selection = X[indices] # fancy indexing here
         selection.shape
```

```
Out[16]: (20, 2)
```

```
In [17]: plt.scatter(X[:, 0], X[:, 1], alpha=0.3)
         plt.scatter(selection[:, 0], selection[:, 1],
         facecolor='none', edgecolor='black', s=200);
```



Sorting array

- `sort()` สำหรับการส่งค่าลำดับข้อมูลภายใน array
- `argsort()` สำหรับการส่งค่า *index* ของลำดับข้อมูลภายใน array

```
In [4]: import numpy as np  
x = np.array([2, 1, 4, 3, 5])  
np.sort(x)
```

```
Out[4]: array([1, 2, 3, 4, 5])
```

```
In [6]: x = np.array([2, 1, 4, 3, 5])  
i = np.argsort(x)  
print(i)
```

```
Out[6]: [1 0 3 2 4]
```

```
In [7]: x[i]
```

```
Out[7]: array([1, 2, 3, 4, 5])
```

Sorting array

- ใช้ argument axis = 0, 1 แทน
การลำดับ row หรือ column

```
In [8]: rng = np.random.default_rng(seed=42)
X = rng.integers(0, 10, (4, 6))
print(X)
```

```
Out[8]: [[0 7 6 4 4 8]
          [0 6 2 0 5 9]
          [7 7 7 7 5 1]
          [8 4 5 3 1 9]]
```

```
In [9]: # sort each column of X
np.sort(X, axis=0)
```

```
Out[9]: array([[0, 4, 2, 0, 1, 1],
               [0, 6, 5, 3, 4, 8],
               [7, 7, 6, 4, 5, 9],
               [8, 7, 7, 7, 5, 9]])
```

```
In [10]: # sort each row of X
np.sort(X, axis=1)
```

```
Out[10]: array([[0, 4, 4, 6, 7, 8],
                [0, 0, 2, 5, 6, 9],
                [1, 5, 7, 7, 7, 7],
                [1, 3, 4, 5, 8, 9]])
```

Pandas



Pandas

- สร้างจากพื้นฐาน Numpy อีกทีเพื่อเพิ่มประสิทธิภาพสำหรับการจัดการข้อมูล
- โครงสร้างข้อมูลที่ยืดหยุ่นได้แก่ Dataframe สามารถดำเนินการแทน array แบบหลายมิติ หรือ Numpy ndarray และ Pandas Series แทน array 1 มิติ
- กำหนดชื่อแถวหรือคอลัมน์ได้
- แต่ละคอลัมน์เป็นข้อมูลต่างชนิดกันได้

```
In [2]: data = pd.Series([0.25, 0.5, 0.75, 1.0])
        data
Out[2]: 0 0.25
        1 0.50
        2 0.75
        3 1.00
        dtype: float64
```

Series กับ Numpy array

- แตกต่างกันที่การสร้าง index
 - Numpy: Integer แบบ implicit
 - Series: ระบุ index ชนิดใดก็ได้เป็น index (explicit)

```
In [7]: data = pd.Series([0.25, 0.5, 0.75, 1.0],  
                        index=['a', 'b', 'c', 'd'])
```

```
data  
Out [7]: a 0.25  
        b 0.50  
        c 0.75  
        d 1.00  
dtype: float64
```

```
In [8]: data['b']
```

```
Out [8]: 0.5
```

Series กับ Dictionary

- สร้าง series จาก dict ได้
- ใช้ index เพื่อลำดับ

```
In [16]: pd.Series({2:'a', 1:'b', 3:'c'})
```

```
Out[16]: 2 a  
         1 b  
         3 c  
dtype: object
```

```
In [17]: pd.Series({2:'a', 1:'b', 3:'c'}, index=[1, 2])
```

```
Out[17]: 1 b  
         2 a  
dtype: object
```

Indexers: loc and iloc

```
In [11]: data = pd.Series(['a', 'b', 'c'],  
                           index=[1, 3, 5])
```

```
data
```

```
Out[11]: 1 a  
         3 b  
         5 c  
dtype: object
```

```
In [12]: # explicit index when indexing  
data[1]
```

```
Out[12]: 'a'
```

```
In [13]: # implicit index when slicing  
data[1:3]
```

```
Out[13]: 3 b  
         5 c  
dtype: object
```

อาจเกิดความสับสน
จึงควรใช้ loc แทน explicit

และ iloc แทน implicit

```
In [14]: data.loc[1]
```

```
Out[14]: 'a'
```

```
In [15]: data.loc[1:3]
```

```
Out[15]: 1 a  
         3 b  
dtype: object
```

DataFrame

- array 2 มิติ
- ใช้ loc/iloc เหมือน series

```
In [18]: area = pd.Series({'California': 423967,  
                           'Texas': 695662,  
                           'Florida': 170312,  
                           'New York': 141297,  
                           'Pennsylvania': 119280})  
  
pop = pd.Series({'California': 39538223,  
                'Texas': 29145505,  
                'Florida': 21538187,  
                'New York': 20201249,  
                'Pennsylvania': 13002700})  
  
data = pd.DataFrame({'area':area, 'pop':pop})  
data
```

```
Out[18]:
```

	area	pop
California	423967	39538223
Texas	695662	29145505
Florida	170312	21538187
New York	141297	20201249
Pennsylvania	119280	13002700

DataFrame – Indexers: loc and iloc

```
In [22]: data['density'] = data['pop'] / data['area']
```

```
Out[22]:
```

	area	pop	density
California	423967	39538223	93.257784
Texas	695662	29145505	41.896072
Florida	170312	21538187	126.463121
New York	141297	20201249	142.970120
Pennsylvania	119280	13002700	109.009893

```
In [27]: data.iloc[:3, :2]
```

```
Out[27]:
```

	area	pop
California	423967	39538223
Texas	695662	29145505
Florida	170312	21538187

```
In [28]: data.loc[:'Florida', :'pop']
```

```
Out[28]:
```

	area	pop
California	423967	39538223
Texas	695662	29145505
Florida	170312	21538187

DataFrame – Indexers: loc and iloc

- สามารถใส่เงื่อนไขการกรองได้

```
In [29]: data.loc[data.density > 120, ['pop', 'density']]
```

```
Out[29]:
```

	pop	density
Florida	21538187	126.463121
New York	20201249	142.970120

```
In [30]: data.iloc[0, 2] = 90
data
```

```
Out[30]:
```

	area	pop	density
California	423967	39538223	90.000000
Texas	695662	29145505	41.896072
Florida	170312	21538187	126.463121
New York	141297	20201249	142.970120
Pennsylvania	119280	13002700	109.009893

Handling Missing Data

- แทนด้วยค่า sentinel
 - ค่าที่มักไม่ใช้เช่น -9999
 - NaN (Not a Number) ค่าพิเศษตาม IEEE floating-point
 - เกิดการลดช่วงการแสดงค่าของข้อมูลลง
 - บางภาษาใช้ NA, (Python, R) สำหรับ nonfloating-point
 - None
- Pandas มีกระบวนการจัดการเบื้องต้น 2 แบบ
 - NaN หรือขึ้นอยู่กับชนิดข้อมูล
 - ใช้ตัวเลือก pd.NA (Nullable)

None as a Sentinel Value

- Numpy ไม่รองรับ None ภายใน array

```
In [1]: import numpy as np
import pandas as pd
```

```
In [2]: vals1 = np.array([1, None, 2, 3])
vals1
```

```
Out[2]: array([1, None, 2, 3], dtype=object)
```

```
In [5]: vals1.sum()
TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
```

NaN: Missing Numerical Data

- Numpy รองรับ NaN เฉพาะฟังก์ชันที่กำหนด

```
In [6]: vals2 = np.array([1, np.nan, 3, 4])  
vals2
```

```
Out[6]: array([ 1., nan, 3., 4.])
```

```
In [9]: vals2.sum(), vals2.min(), vals2.max()
```

```
Out[9]: (nan, nan, nan)
```

```
In [10]: np.nansum(vals2), np.nanmin(vals2), np.nanmax(vals2)
```

```
Out[10]: (8.0, 1.0, 4.0)
```

NaN and None in Pandas

- Pandas รองรับ None และ NaN แบบปะปนกันได้ แต่ถ้าระบุชนิดตัวแปร จะถูกทำให้เป็นมาตรฐานเดียว

```
In [1]: pd.Series([1, np.nan, 2, None, pd.NA], dtype='Int32')
```

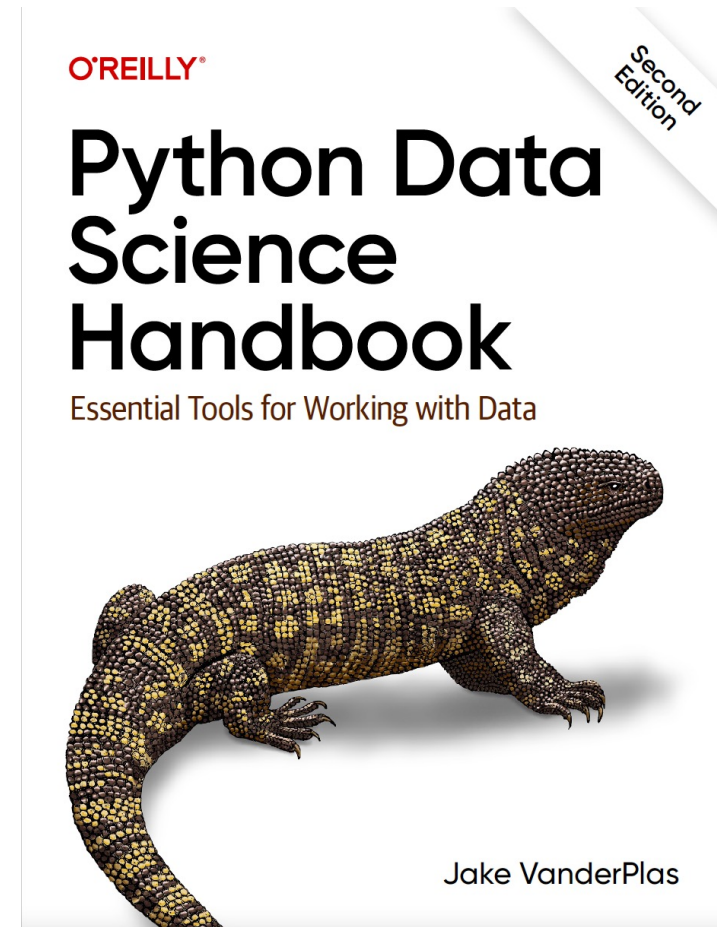
```
Out[1]: 0 1  
1 <NA>  
2 2  
3 <NA>  
4 <NA>  
dtype: Int32
```

```
In [2]: pd.Series([1, np.nan, 2, None, pd.NA])
```

```
Out[2]: 0 1  
1 NaN  
2 2  
3 None  
4 <NA>  
dtype: object
```

เรียนรู้จากตัวอย่างเพิ่มเติม

- Chapter 16: Handling Missing Data
- Chapter 17: Hierarchical Indexing
- Chapter 18: Combining Datasets: concat and append
- Chapter 19: Combining Datasets: merge and join
- Chapter 20: Aggregation and Grouping
- Chapter 21: Pivot Tables
- Chapter 22: Vectorized String Operations
- Chapter 23: Working with Time Series



การรวมและการจัดกลุ่ม

Aggregation and Grouping

Table 20-1. Listing of Pandas aggregation methods

Aggregation	Returns
count	Total number of items
first, last	First and last item
mean, median	Mean and median
min, max	Minimum and maximum
std, var	Standard deviation and variance
mad	Mean absolute deviation
prod	Product of all items
sum	Sum of all items

```
In [7]: df = pd.DataFrame({'A': rng.rand(5),  
                           'B': rng.rand(5)})
```

```
Out[7]:
```

	A	B
0	0.155995	0.020584
1	0.058084	0.969910
2	0.866176	0.832443
3	0.601115	0.212339
4	0.708073	0.181825

```
In [8]: df.mean()
```

```
Out[8]:
```

A	0.477888
B	0.443420

```
dtype: float64
```

```
In [9]: df.mean(axis='columns')
```

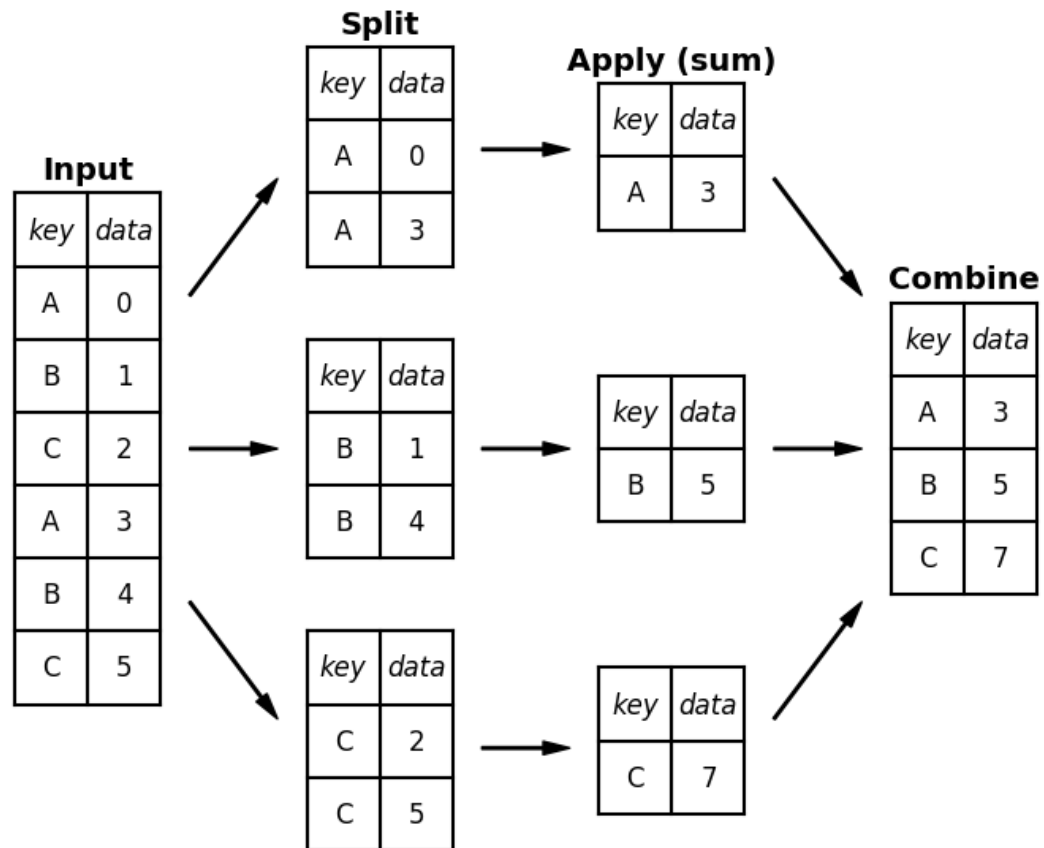
```
Out[9]:
```

0	0.088290
1	0.513997
2	0.849309
3	0.406727
4	0.444949

```
dtype: float64
```

groupby: Split, Apply, Combine

```
In [11]: df = pd.DataFrame({'key': ['A', 'B', 'C', 'A', 'B', 'C'],  
                           'data': range(6)}, columns=['key', 'data'])
```



```
In [13]: df.groupby('key').sum()  
Out[13]: key    data  
A         3  
B         5  
C         7
```

Aggregation

```
In [19]: rng = np.random.RandomState(0)
df = pd.DataFrame({'key': ['A', 'B', 'C', 'A', 'B', 'C'],
                  'data1': range(6),
                  'data2': rng.randint(0, 10, 6)},
                  columns = ['key', 'data1', 'data2'])
```

```
Out[19]: df
```

	key	data1	data2
0	A	0	5
1	B	1	0
2	C	2	3
3	A	3	3
4	B	4	7
5	C	5	9

```
In [20]: df.groupby('key').aggregate(['min', np.median, max])
```

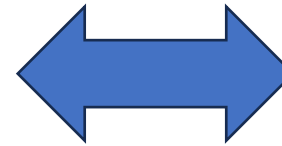
```
Out[20]:
```

	data1			data2		
	min	median	max	min	median	max
key						
A	0	1.5	3	3	4.0	5
B	1	2.5	4	0	3.5	7
C	2	3.5	5	3	6.0	9

Pivot Tables

รูปแบบ Pivot ของ MS-Excel

State	Rank	Index	Grocery	Housing	Utilities	Transportation	Health	Misc.
Mississippi	1	84.9	92.6	68.5	89.7	93.3	99.6	91.5
Oklahoma	2	85.7	93.3	71	95.2	90.6	95.5	90.7
Kansas	3	86.4	92	71.7	98.1	97.1	102.7	89.1
Alabama	4	87.5	97.8	69.9	100.7	90	89.6	94.2
Georgia	5	87.8	93.6	75.4	91	88.3	95.2	95
West Virginia	6	88.5	98.7	67.8	94.5	111	100.6	94.5
Missouri	7	88.6	95.9	79.5	95.3	90.2	91.7	90.6
Indiana	8	88.9	93.3	77.3	103.1	91.3	95.6	91.9
Iowa	9	89	101	72	94	96.4	99.4	94.5
Tennessee	10	89.5	94.2	80	94	90.8	89.9	94.8
Arkansas	11	89.9	92.9	78.1	96.9	92.8	81.2	98.8
Texas	12	90.9	89.9	82.5	100.5	93.4	94.4	95.9
Illinois	13	91.4	98	80.7	92.2	105.4	97.8	94
Nebraska	14	91.5	97.7	83.6	86.5	95.1	100	95.5
Michigan	15	91.6	91.7	80.6	99.1	99	97.8	97.6
Minnesota	16	91.8	102	80.4	82.2	92.2	100.1	98.5



State	Category	CostOfLiving
Arkansas	Grocery	92.9
Arkansas	Health	81.2
Arkansas	Housing	78.1
Arkansas	Misc.	98.8
Arkansas	Transportation	92.8
Arkansas	Utilities	96.9
Arkansas	Index	89.9
Arkansas	Rank	11
Texas	Grocery	89.9
Texas	Health	94.4
Texas	Housing	82.5
Texas	Misc.	95.9
Texas	Transportation	93.4
Texas	Utilities	100.5
Texas	Index	90.9
Texas	Rank	12
Illinois	Grocery	98
Illinois	Health	97.8

Example: Birthrate Data

```
In [11]: births = pd.read_csv('data/births.csv')
```

```
In [12]: births.head()
```

```
Out[12]:
```

	year	month	day	gender	births
0	1969	1	1.0	F	4046
1	1969	1	1.0	M	4440
2	1969	1	2.0	F	4454
3	1969	1	2.0	M	4548
4	1969	1	3.0	F	4548

```
In [13]: births['decade'] = 10 * (births['year'] // 10)
births.pivot_table('births', index='decade', columns='gender',
                    aggfunc='sum')
```

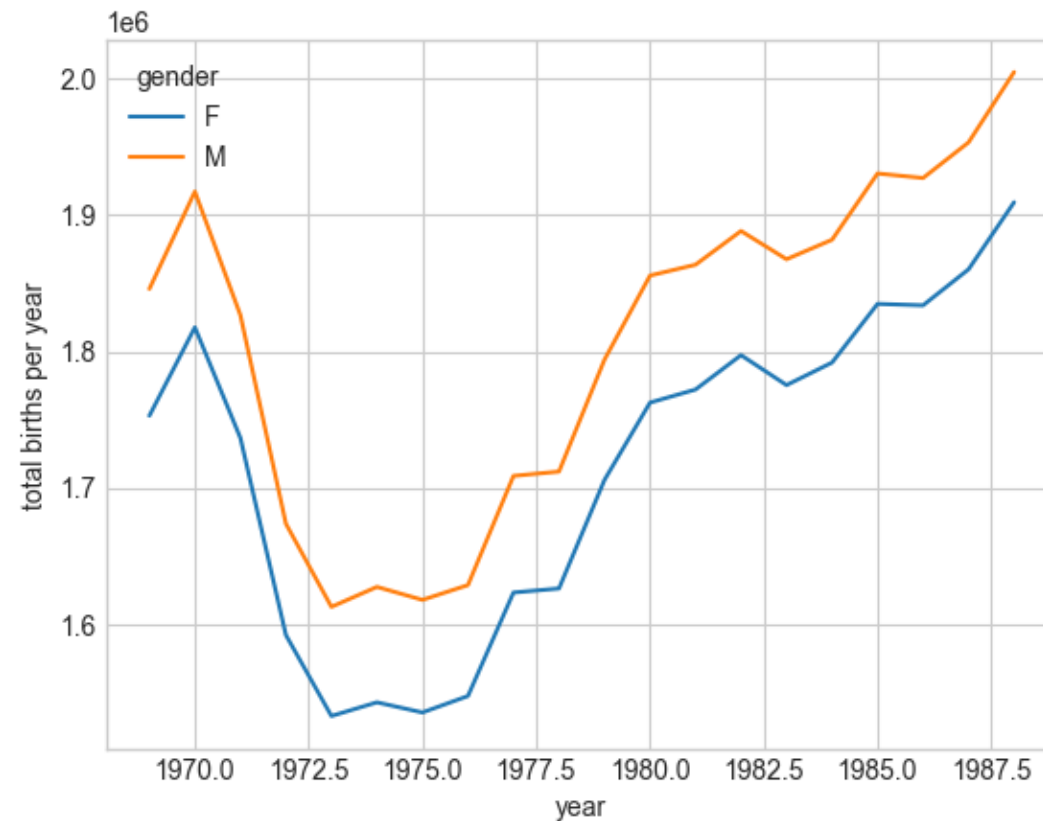
```
Out[13]:
```

	gender F	M
decade		
1960	1753634	1846572
1970	16263075	17121550
1980	18310351	19243452
1990	19479454	20420553
2000	18229309	19106428

Example: Birthrate Data

```
In [14]: %matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('seaborn-whitegrid')
births.pivot_table('births', index='year', columns='gender',
                    aggfunc='sum').plot()

plt.ylabel('total births per year');
```



การดำเนินการ String

- ใช้ Python String Methods ได้เช่นเดียวกัน

len()	lower()	translate()	islower()
ljust()	upper()	startswith()	isupper()
rjust()	find()	endswith()	isnumeric()
center()	rfind()	isalnum()	isdecimal()
zfill()	index()	isalpha()	split()
strip()	rindex()	isdigit()	rsplit()
rstrip()	capitalize()	isspace()	partition()
lstrip()	swapcase()	istitle()	rpartition()

```
In [1]: monte = pd.Series(['Graham Chapman', 'John Cleese', 'Terry Gilliam',  
                           'Eric Idle', 'Terry Jones', 'Michael Palin'])
```

```
In [2]: monte.str.split()
```

```
Out[2]: 0 [Graham, Chapman]  
1 [John, Cleese]  
2 [Terry, Gilliam]  
3 [Eric, Idle]  
4 [Terry, Jones]  
5 [Michael, Palin]  
dtype: object
```

การดำเนินการ String: Regular Expressions

- หาความเข้ากันกับ RE

Method	Description
match	Calls <code>re.match</code> on each element, returning a Boolean.
extract	Calls <code>re.match</code> on each element, returning matched groups as strings.
findall	Calls <code>re.findall</code> on each element
replace	Replaces occurrences of pattern with some other string
contains	Calls <code>re.search</code> on each element, returning a boolean
count	Counts occurrences of pattern
split	Equivalent to <code>str.split</code> , but accepts regexps
rsplit	Equivalent to <code>str.rsplit</code> , but accepts regexps

```
In [1]: monte = pd.Series(['Graham Chapman', 'John Cleese', 'Terry Gilliam',  
                           'Eric Idle', 'Terry Jones', 'Michael Palin'])  
  
In [11]: monte.str.findall(r'^[^AEIOU].*[^aeiou]$')  
Out[11]: 0 [Graham Chapman]  
          1 []  
          2 [Terry Gilliam]  
          3 []  
          4 [Terry Jones]  
          5 [Michael Palin]  
dtype: object
```

ข้อมูลอนุกรมเวลา (Time Series)

- Timestamps ระบุเวลาขณะบันทึกข้อมูล เช่น July 4, 2021 at 7:00 a.m.
- Time intervals and periods ระยะเวลาตามช่วงที่กำหนด เช่น ปี 1970 – 2005
- Time deltas or durations เเจาะจงช่วงเวลาใดๆ เช่น 22 วินาที

*** Unix Timestamp เวลามาตรฐานที่ใช้ในการเปรียบเทียบทั่วโลก ตัวเลขระบุเวลาที่บวก 1 เพิ่มทุกๆ วินาที เริ่มนับตั้งแต่วินาทีที่ 00:00:00 ของวันพฤหัสบดี วันที่ 1 มกราคม ค.ศ.1970 เป็นต้นมาตามเวลาสากล UTC

โครงสร้างข้อมูล Pandas Time Series

```
In [15]: dates = pd.to_datetime([datetime(2021, 7, 3), '4th of July, 2021',  
                                '2021-Jul-6', '07-07-2021', '20210708'])
```

```
dates  
Out[15]: DatetimeIndex(['2021-07-03', '2021-07-04', '2021-07-06',  
                        '2021-07-07', '2021-07-08'],  
dtype='datetime64[ns]', freq=None) # ns = nano second
```

```
In [16]: dates.to_period('D')
```

```
Out[16]: PeriodIndex(['2021-07-03', '2021-07-04', '2021-07-06', '2021-07-07',  
                     '2021-07-08'],  
dtype='period[D]') # D = Daily
```

```
In [17]: dates - dates[0]
```

```
Out[17]: TimedeltaIndex(['0 days', '1 days', '3 days', '4 days', '5 days'],  
dtype='timedelta64[ns]', freq=None)
```

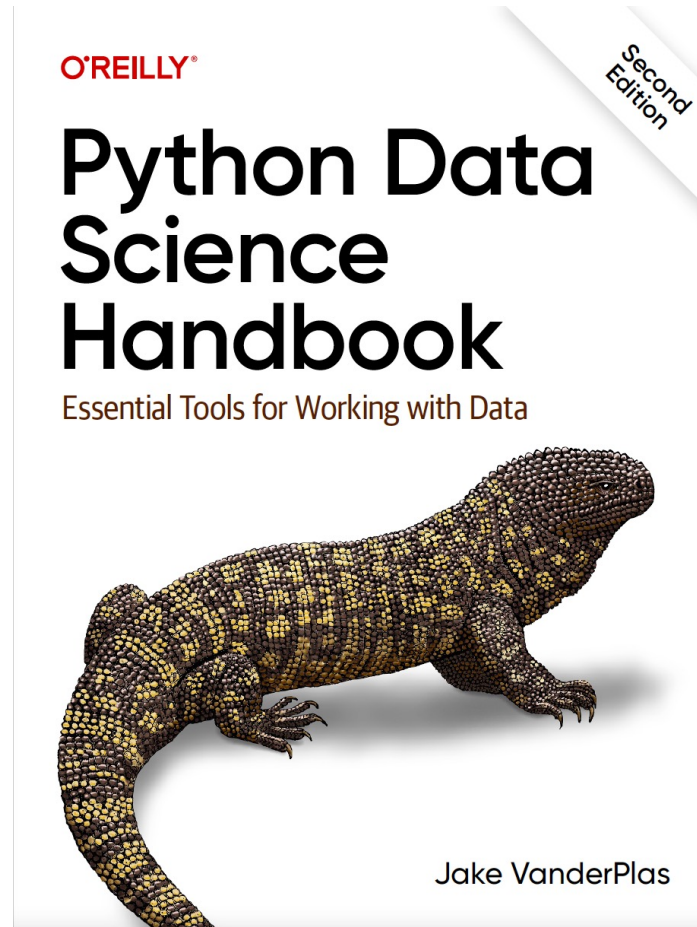
โครงสร้างข้อมูล Pandas Time Series: Frequency codes

```
In [19]: pd.date_range('2015-07-03', periods=8)
Out[19]: DatetimeIndex(['2015-07-03', '2015-07-04', '2015-07-05', '2015-07-06',
                        '2015-07-07', '2015-07-08', '2015-07-09', '2015-07-10'],
                        dtype='datetime64[ns]', freq='D')
```

```
In [20]: pd.date_range('2015-07-03', periods=8, freq='H')
Out[20]: DatetimeIndex(['2015-07-03 00:00:00', '2015-07-03 01:00:00',
                        '2015-07-03 02:00:00', '2015-07-03 03:00:00',
                        '2015-07-03 04:00:00', '2015-07-03 05:00:00',
                        '2015-07-03 06:00:00', '2015-07-03 07:00:00'],
                        dtype='datetime64[ns]', freq='H')
```

```
In [21]: pd.period_range('2015-07', periods=8, freq='M')
Out[21]: PeriodIndex(['2015-07', '2015-08', '2015-09',
                      '2015-10', '2015-11', '2015-12',
                      '2016-01', '2016-02'],
                      dtype='period[M]')
```

Ref



<https://github.com/jakevdp/PythonDataScienceHandbook>

Q & A